

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе № 3

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

Контейнеризация сервиса «Lecter's Recipes» средствами Docker

Вариант 5

Выполнил: студент группы БР1.1
Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

Контейнеризовать приложение: написать Dockerfile для каждого сервиса, собрать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами.

2. Образы сервисов

Для каждого сервиса написан многоэтапный Dockerfile. На этапе сборки ставятся все зависимости и компилируется TypeScript; в итоговый образ попадают только каталог dist и production-зависимости. Приложение выполняется от непривилегированного пользователя node, встроенного в базовый образ.

```
# ----- этап сборки -----
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

# ----- этап выполнения -----
FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
RUN apk add --no-cache wget
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=builder /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Листинг 1 — Dockerfile сервиса (многоэтапная сборка)

Образ шлюза дополнительно получает каталоги public и docs — статическую витрину и спецификацию OpenAPI. Файл .dockerignore исключает node_modules, dist, .env и логи, что ускоряет сборку и не допускает попадания секретов в образ.

3. Состав docker-compose

Таблица 1 — Состав контура docker compose

Контейнер	Образ или сборка	Порт наружу	Роль
lecters-postgres	postgres:17-alpine	5433 → 5432	три базы данных, том postgres-data

Контейнер	Образ или сборка	Порт наружу	Роль
lecters-rabbitmq	rabbitmq:4-management-alpine	5672, 15672	брокер сообщений, том rabbitmq-data
lecters-user-service	сборка ../lab2/user-service	—	учётные записи и подписки
lecters-recipe-service	сборка ../lab2/recipe-service	—	рецепты и справочники
lecters-social-service	сборка ../lab2/social-service	—	реакции и комментарии
lecters-gateway	сборка ../lab2/api-gateway	3000	единственная публичная точка входа

Сборка ведётся напрямую из каталогов ЛР2, поэтому исходный код не дублируется: контейнеризация не создаёт второй копии приложения.

4. Сетевое взаимодействие

Все контейнеры включены в выделенную сеть **lecters** с драйвером **bridge**. Внутри сети сервисы обращаются друг к другу по именам контейнеров, которые Docker разрешает через встроенный DNS:

```

USER_SERVICE_URL: http://user-service:3001
RECIPE_SERVICE_URL: http://recipe-service:3002
SOCIAL_SERVICE_URL: http://social-service:3003
RABBITMQ_URL: amqp://lecters:lecters@rabbitmq:5672
DB_HOST: postgres

```

Листинг 2 — адресация сервисов внутри сети контура

Наружу опубликован только шлюз; порты сервисов не пробрасываются на хост. Порт PostgreSQL проброшен на 5433, чтобы не конфликтовать с СУБД, установленной на машине разработчика, а порт 15672 открыт для веб-консоли брокера. Такая схема делает внутренние эндпоинты `/internal` недоступными извне на уровне сети, а не только на уровне маршрутизации.

5. Порядок запуска и проверки состояния

Для каждого контейнера описана проверка состояния, а зависимости объявлены условием `service_healthy`. Благодаря этому сервисы не стартуют раньше, чем база примет подключения, а шлюз — раньше, чем поднимутся все три сервиса.

```

social-service:
  depends_on:
    postgres:      { condition: service_healthy }
    rabbitmq:      { condition: service_healthy }
    recipe-service: { condition: service_healthy }
  healthcheck:
    test: ["CMD", "wget", "-qO-", "http://localhost:3003/health"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 15s

```

Листинг 3 — зависимости и проверка состояния контейнера

6. Инициализация баз данных

Скрипт `postgres/init-databases.sh` монтируется в каталог автозапуска образа и создаёт три базы — по одной на сервис. Локаль `en_US.utf8` задана осознанно: в базе с локалью `C` оператор `ILIKE` не сворачивает регистр кириллицы, из-за чего перестает работать поиск рецептов по названию — дефект, найденный ещё в ЛР1.

```

for db in lecters_users lecters_recipes lecters_social; do
  psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname postgres <<-EOSQL
    SELECT 'CREATE DATABASE $db TEMPLATE template0 ENCODING 'UTF8'
          LC_COLLATE 'en_US.utf8' LC_CTYPE 'en_US.utf8'
          WHERE NOT EXISTS (SELECT FROM pg_database WHERE datname = '$db')\gexec
EOSQL
done

```

Листинг 4 — создание баз по сервисам при первом запуске

7. Конфигурация и секреты

Значения настроек вынесены в переменные окружения с запасными значениями по умолчанию (синтаксис `${VAR:-default}`), а файл `.env.example` показывает полный перечень. Сами файлы `.env` исключены из репозитория и из образов. Общие для контура секреты — `JWT_SECRET` и `INTERNAL_TOKEN` — задаются в одном месте и раздаются всем сервисам, что исключает рассинхронизацию.

8. Запуск контура

```

$ docker compose up -d --build
$ docker compose ps
NAME                                STATUS                                PORTS
lecters-gateway                     Up 10 seconds (healthy)              0.0.0.0:3000->3000/tcp
lecters-postgres                    Up 2 minutes (healthy)              0.0.0.0:5433->5432/tcp
lecters-rabbitmq                     Up 2 minutes (healthy)              0.0.0.0:5672->5672/tcp, 0.0.0.0:15672->15672/tcp
lecters-recipe-service              Up 2 minutes (healthy)              3000/tcp
lecters-social-service              Up 2 minutes (healthy)              3000/tcp
lecters-user-service                Up 2 minutes (healthy)              3000/tcp

$ curl -s localhost:3000/health
{
  "status": "ok",
  "service": "api-gateway",
  "services": {
    "user-service": { "status": "ok", "broker": "connected" },
    "recipe-service": { "status": "ok", "broker": "connected" },
    "social-service": { "status": "ok", "broker": "connected",
      "projections": { "recipes": 6, "users": 4 } }
  }
}

```

Листинг 5 — запуск контура и проверка состояния

Изоляция сети подтверждается напрямую: внутренние эндпоинты отвечают только внутри контура и только с общим секретом, а с хоста порты сервисов недоступны.

```

$ docker compose exec gateway wget -qO- http://user-service:3001/internal/users?ids=1
HTTP/1.1 401 Unauthorized

$ docker compose exec gateway wget -qO- \
  --header="X-Internal-Token: lecters-internal-token" \
  "http://user-service:3001/internal/users?ids=1,2"
{"data":[{"id":1,"username":"dr_lecter",...}, {"id":2,"username":"jack_crawford",...}]

$ curl -s -o /dev/null -w "%{http_code}\n" "http://localhost:3000/internal/users?ids=1"
404

$ curl -s --max-time 3 http://localhost:3001/health
(порт 3001 с хоста недоступен — сервисы скрыты за шлюзом)

```

Листинг 6 — проверка сетевой изоляции внутренних эндпоинтов

9. Проверка контура в контейнерах

После запуска в контейнерах выполнены те же сценарии, что и при локальной разработке: интеграционный сценарий ЛР2, проверка событий Д35 и коллекция Postman из Д33. Все прогоны успешны, что подтверждает работоспособность контейнеризованной версии.

Таблица 2 — Проверки, выполненные на контейнеризованном контуре

Проверка	Команда	Результат
Состояние контура	curl localhost:3000/health	status: ok, три сервиса, брокер подключён
События (Д35)	node homeworks/hw5/events-smoke.mjs	22 успешно, 0 провалено

Проверка	Команда	Результат
Сценарий Postman (Д33)	<code>newman run ... --env-var baseUrl=...</code>	28 запросов, 75 проверок, 0 ошибок
Витрина и Swagger	браузер: / и /docs	страницы отдаются шлюзом из образа

10. Выводы

Для каждого сервиса подготовлен многоэтапный Dockerfile, дающий компактный образ без исходников и dev-зависимостей и запускающий приложение от непривилегированного пользователя. Общий docker-compose.yml поднимает весь контур одной командой: PostgreSQL с тремя базами, RabbitMQ, три сервиса и шлюз.

Сетевое взаимодействие настроено через выделенную сеть и DNS-имена контейнеров; наружу опубликован только шлюз, что заодно закрывает внутренние эндпоинты от внешнего доступа. Порядок запуска обеспечен проверками состояния, а данные сохраняются в именованных томах между перезапусками.